FinVault – Stock Trading Platform

Project Documentation

1. INTRODUCTION

Project Title:	**FinVault - Stock Trading Platform**
Team Configuration:	Individual Project
Developer:	Ganesh Basode
Institution:	G. Pullaiah College of Engineering and Technology
Department:	Computer Science and Engineering

2. PROJECT OVERVIEW

Purpose

FinVault is a scalable web-based platform developed using the MERN stack. It provides users with a realistic experience of stock market operations by using virtual funds instead of real money, allowing them to practice investment strategies without financial risk. The platform supports simulated stock buying and selling, portfolio monitoring, watchlist management, balance tracking, and administrative controls through an interactive environment.

Core Feature Matrix

- Authentication Gateway: Provides secure user registration, login functionality, and session management using JSON Web Token (JWT)-based authentication to protect user accounts.
- Order Processing Simulation: Validates transactions by checking available virtual funds and processes simulated stock purchase and selling operations efficiently.
- Transaction History & Data Management: Maintains detailed records of user transactions, stock prices, trading activities, and portfolio changes using a structured database system.
- Admin Control Panel: Enables administrators to manage user accounts, update stock listings, monitor platform activities, and perform database management operations.

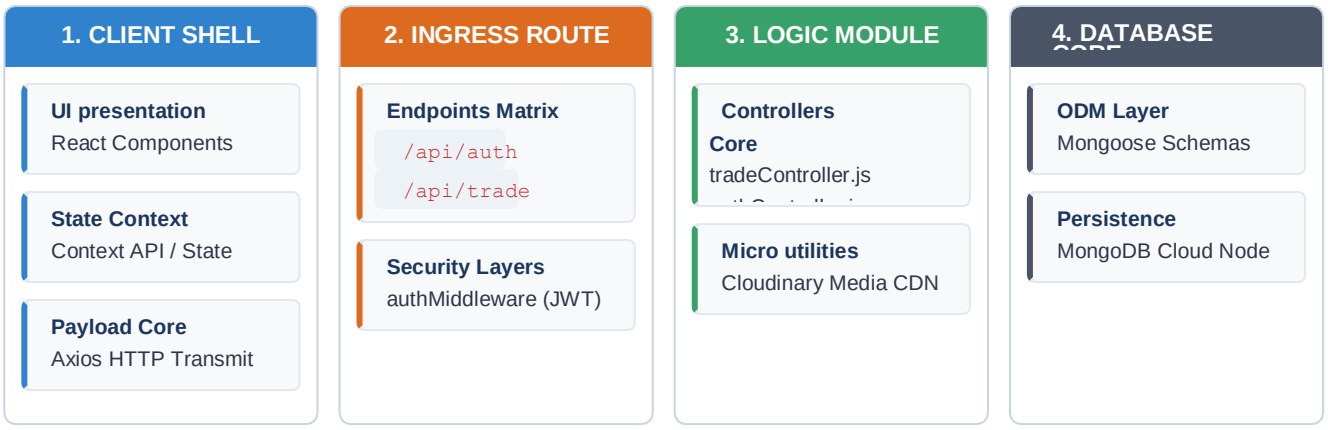
3. SYSTEM ARCHITECTURE

Frontend Tier

The frontend of FinVault is developed as a Single Page Application (SPA) using **React.js** with the **Vite** build tool for efficient development and optimized performance. The application follows a component-based structure where data and state management are handled using React Context API and local state hooks. Communication between the frontend and backend is managed through Axios, enabling smooth exchange of data through asynchronous API requests.

Backend Tier

The backend system follows a structured layered architecture built using **Node.js** and **Express.js**. Incoming client requests are processed through defined API routes, where authentication and validation checks are performed before executing the required business logic. Dedicated controller modules handle application operations, interact with the database, and return processed responses in a structured JSON format.



Database Architecture

Collection Store	Mongoose File Node	Data Document Attributes
Users	`User.js`	Profile details, encrypted security parameters, wallet balances, and user role claims.
Stocks	`stock.js`	Asset tickers, corporate names, current market tracking price fields, and historical price matrices.
Transactions	`transaction.js`	User entity links, stock IDs, execution type metrics, transacted prices, and timestamps.

4. SETUP & DEPLOYMENT INSTRUCTIONS

4.1 Prerequisites

Software Component	System Deployment Purpose
Node.js	Core backend runtime framework platform.
npm	Global library manager tracking setup dependencies.
MongoDB Atlas	Cloud-managed database host managing application records.
Visual Studio Code	Primary development IDE environment.

4.2 Installation Pipeline

```
  # 1. Clone repository and initialize server
steps cd backend
  npm install

  # 2. Open an independent tab to initialize client
components cd frontend
  npm install

  # 3. Establish environment configurations matching target cluster
properties # Setup .env file inside the backend root path folder
```

4.3 Environment Variables

```
  PORT=5001
  MONGO_URI=mongodb+srv://<username>:<password>@cluster0.atlas.mongodb.net/FinVault
JWT_SECRET=your_configured_secure_secret_key_string
```

5. PROJECT FOLDER ARCHITECTURE

```
FinVault/
├── frontend/ — Client Interface Layer
│ └── src/
│ ├── admin/ — Administrative view layouts
│ ├── api/ — Axios service network calls
│ ├── assets/ — Static visual resources and images
│ ├── components/ — Reusable structural layout blocks
│ ├── context/ — React Authentication and Session contexts
│ ├── css/ — Custom interface stylesheets
│ ├── pages/ — Top-level view screen routers
│ ├── App.jsx — Primary structural page node entry
│ └── main.jsx — DOM initialization baseline code
└── backend/ — Server Infrastructure Layer
├── config/ — DB configuration files and handshake systems
├── controllers/ — Core functional computation business logic
├── middleware/ — JWT session checkers, admin filters, file loaders
├── models/ — Mongoose structural collection definitions
├── routes/ — Express isolated REST API endpoints
├── uploads/ — Storage path for uploaded asset images
│ ├── 1782888122118.jpg
│ └── 1782888728837.jpg
├── scripts/ — Utility and programmatic automation scripts
├── seed/ — Hardcoded initial baseline database mock entries
├── .env — Secure environment parameters container file
├── package-lock.json — Fixed precise package structural tree map
├── package.json — Primary script listings and engine specifications
├── resetData.js — Global database records scrubbing script
├── resetPortfolio.js — User portfolio initialization script
├── seedStocks.js — Mock market stocks generator seed script
└── server.js — Primary Node application entry gateway bootstrap
```

6. INITIALIZING & RUNNING THE APPLICATION

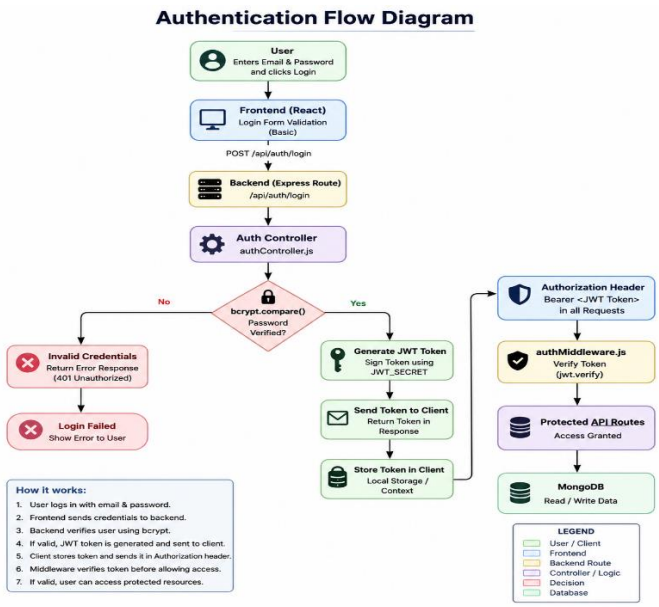
```
  # Launch Server Instance (Window
1) cd backend && node server.js



  # Launch Client Development Server (Window
2) cd frontend && npm run dev
```

7. API ENDPOINT SPECIFICATIONS

Method	Endpoint Node Route	Description	Auth Level
POST	/api/auth/register	Registers a new profile asset document inside user clusters.	No Auth
POST	/api/auth/login	Validates profile credentials to issue matching session JWT key.	No Auth
GET	/api/auth/me	Pulls current profile attributes tracking current sessions.	JWT User
GET	/api/stocks	Pulls global stocks indices records.	JWT User
POST	/api/trade/buy	Initializes secure virtual balance matching buy transaction sequences.	JWT User
POST	/api/trade/sell	Initializes virtual share validation liquidation transaction sequences.	JWT User
GET	/api/admin/users	Administrative access path to map active user account registries.	Admin Only

8. AUTHENTICATION & SECURITY

Incoming access requests are validated through secure authentication mechanisms. User passwords are encrypted using bcrypt hashing before being stored in the database. After successful login verification, unique authentication tokens are generated and returned to authorize subsequent requests to protected resources.

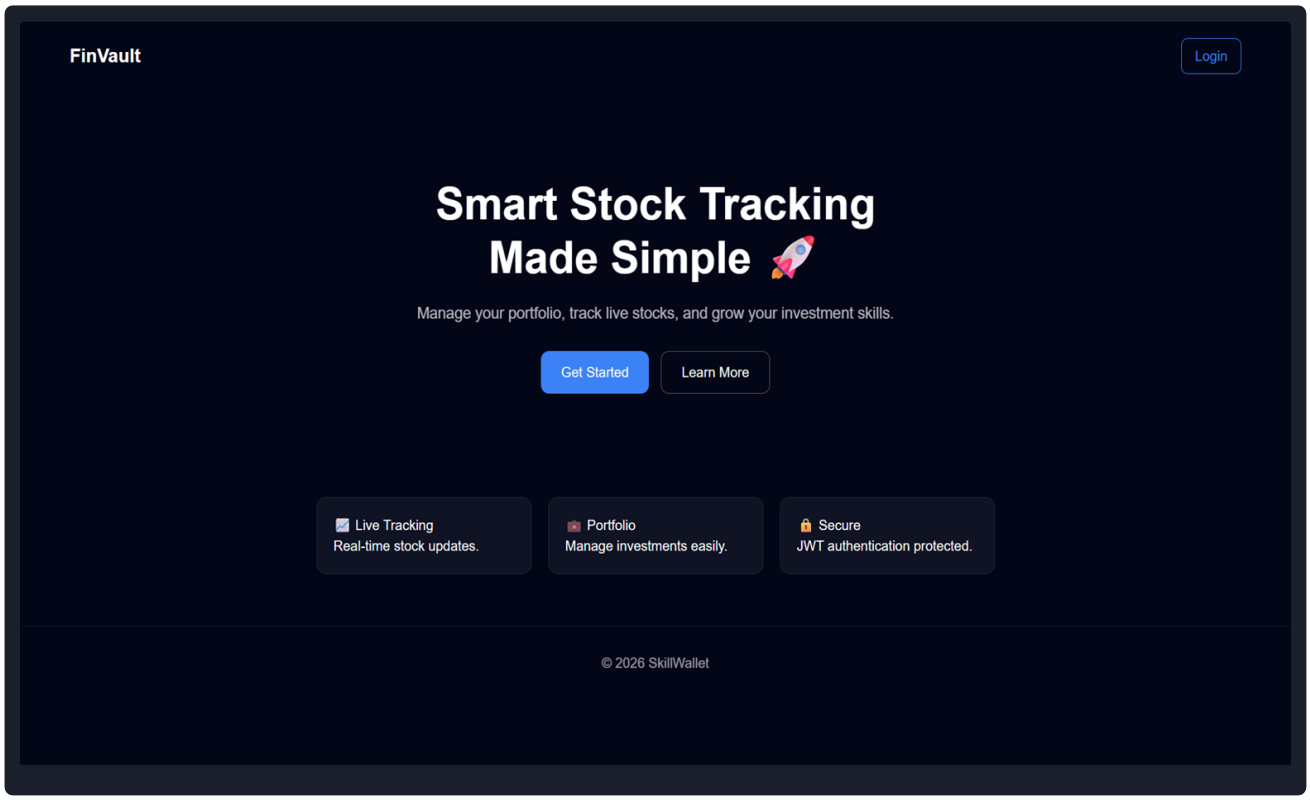


9. USER INTERFACE

This section presents the graphical user interface of the Stock Trading Application. The following screenshots illustrate the key application pages and their functionalities, providing an overview of the user experience and system features:

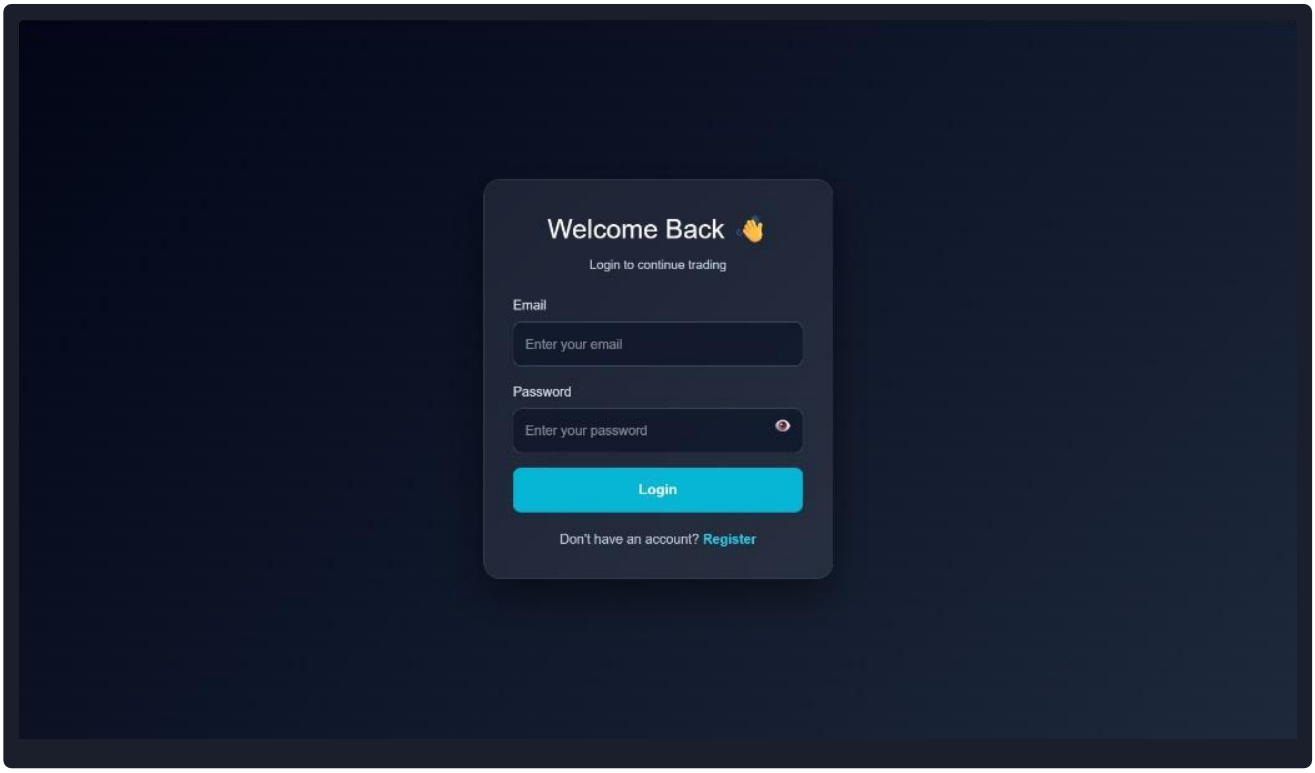
9.1 Landing Page

Serves as the entry point of the stock trading platform, providing an overview of its features and navigation options. It allows users to access the Login and Registration pages to begin using the application.



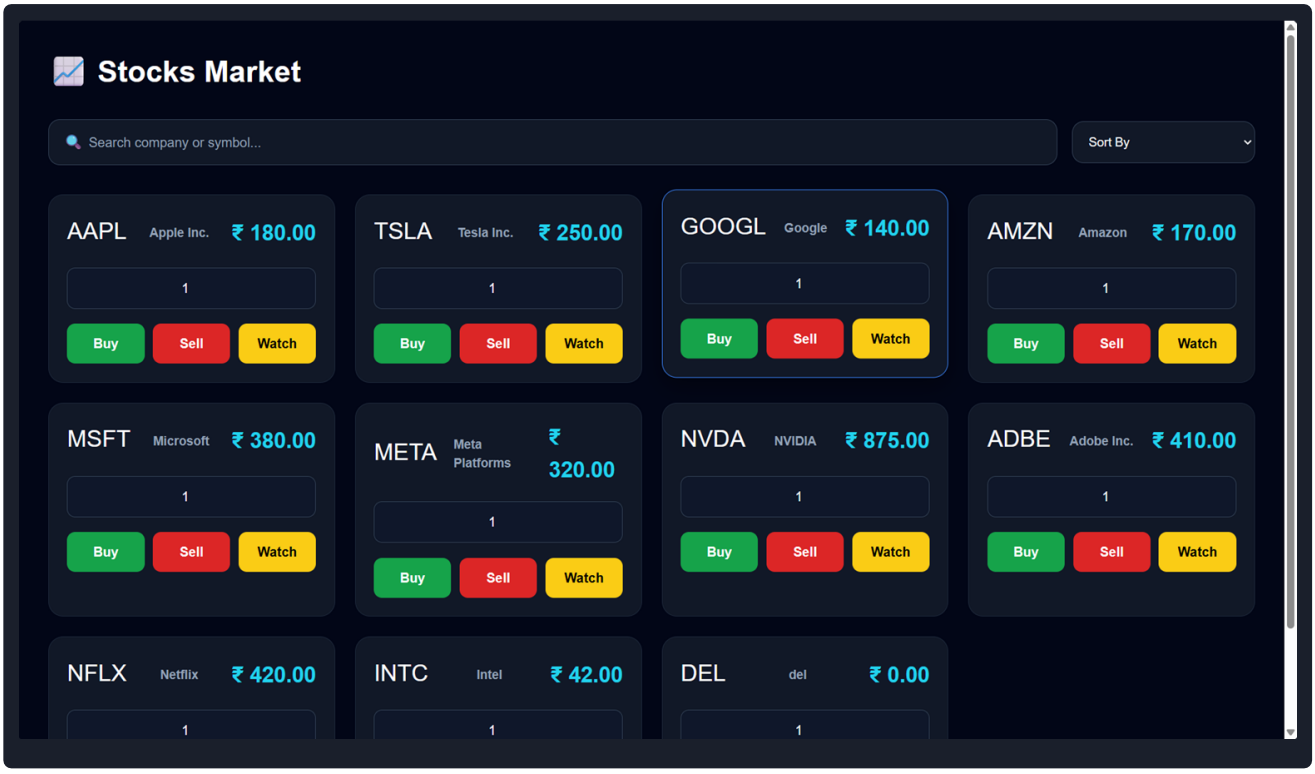
9.2 Login Page

Allows registered users to securely log in using their email and password. After successful authentication, users are redirected to the trading dashboard to access the platform.



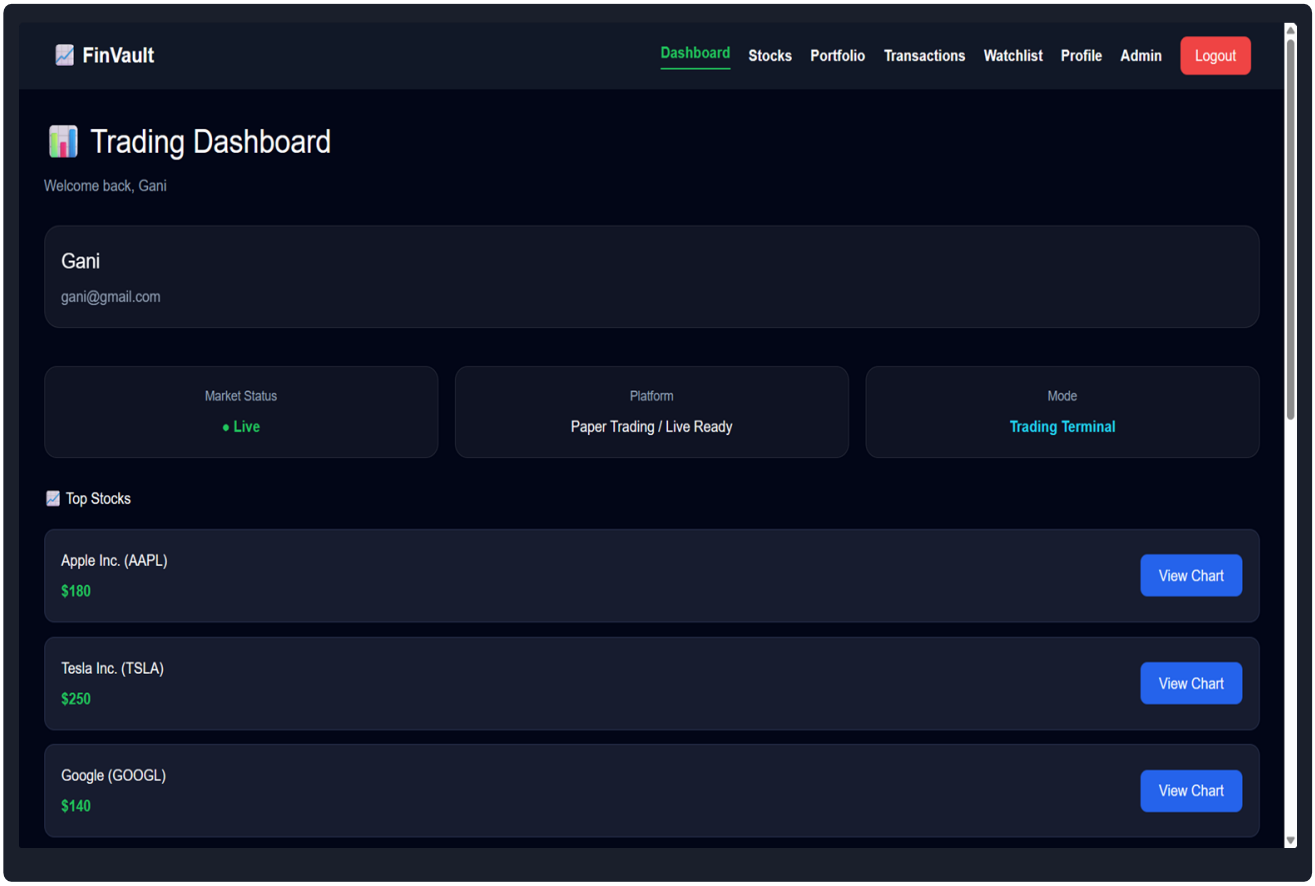
9.3 Stock Market

Displays the list of available stocks with their current prices. Users can buy, sell, add stocks to the watchlist, and view stock details for trading.



9.4 Trading Dashboard

Provides the main interface of the stock trading platform with navigation to Dashboard, Stocks, Portfolio, Transactions, Watchlist, Profile, Admin, and Logout. Displays the user's details, market status, live platform indicators, paper trading mode, trading terminal, and stock charts for quick access to trading information.

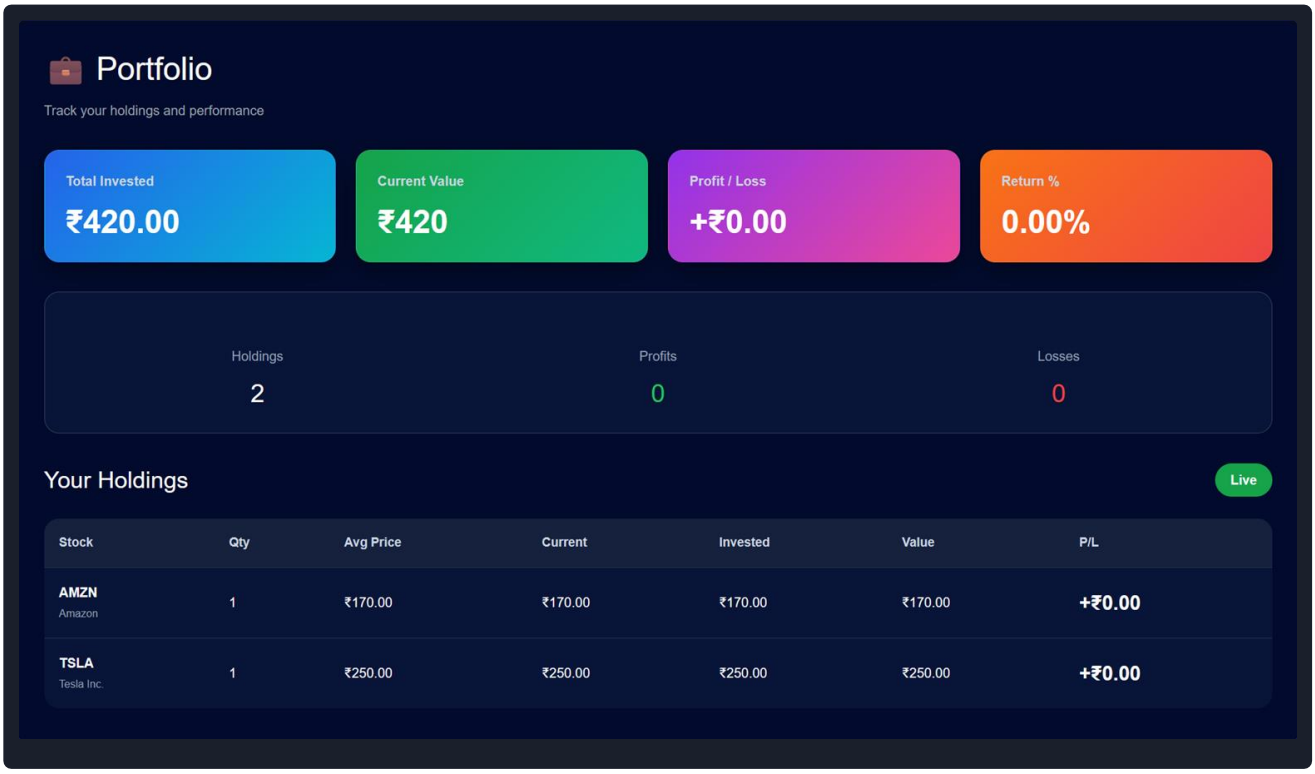


9.5 Stock Analytics

Provides an interactive chart to visualize stock price movements. It also displays the stock's Open, High, Low, and Close (OHLC) values, allowing users to analyze price trends and market performance.

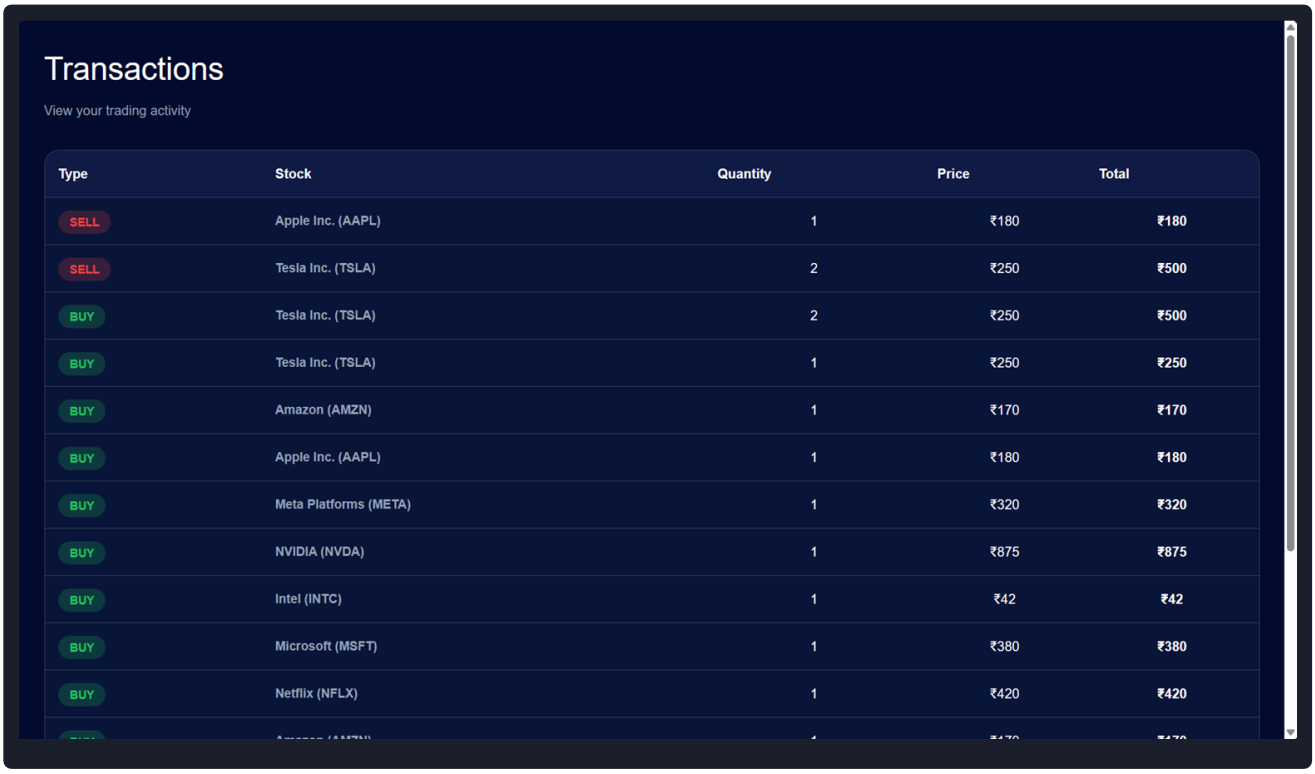
9.6 Portfolio

Provides a summary of the user's stock holdings, showing investment details, current value, and profit or loss for each stock. This allows users to track the performance of their portfolio easily.



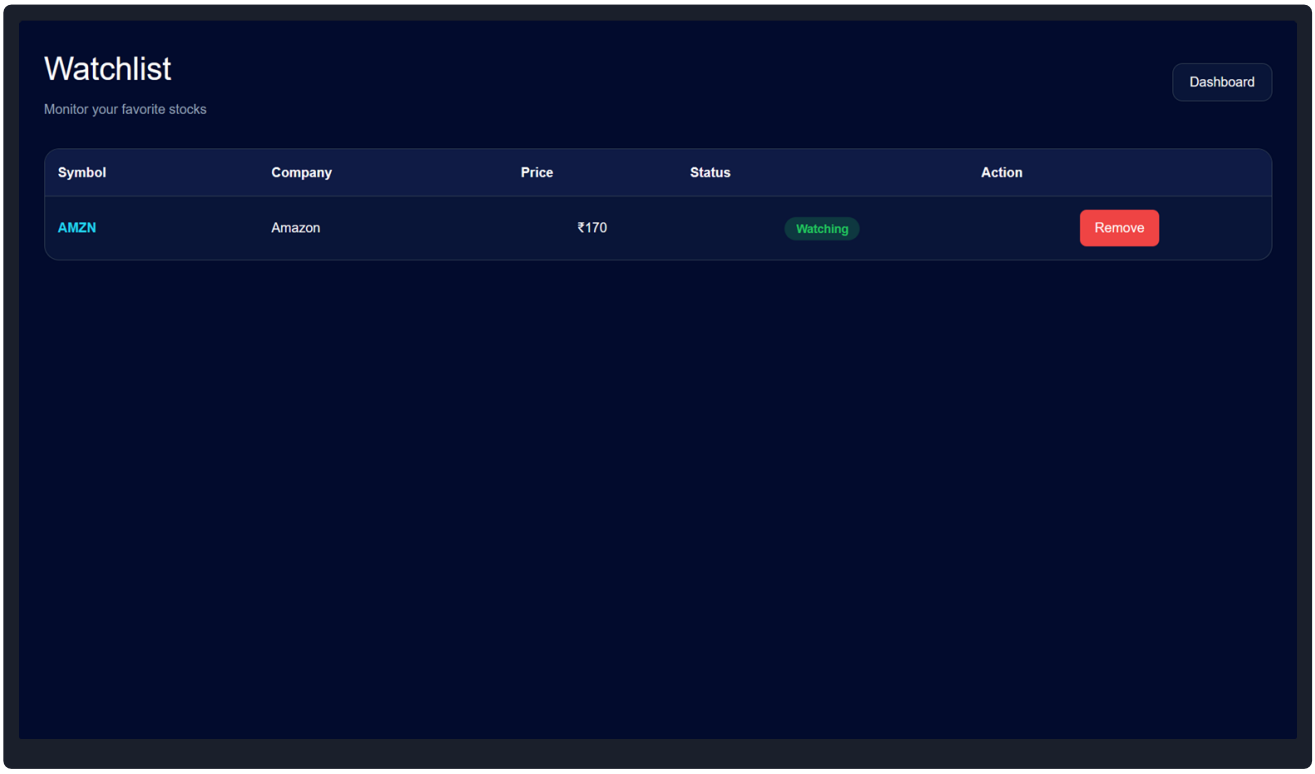
9.7 Transaction History

Provides users with a detailed record of all buy and sell transactions, including stock details, quantity, price, total amount, and transaction date. This enables users to review their trading history easily.



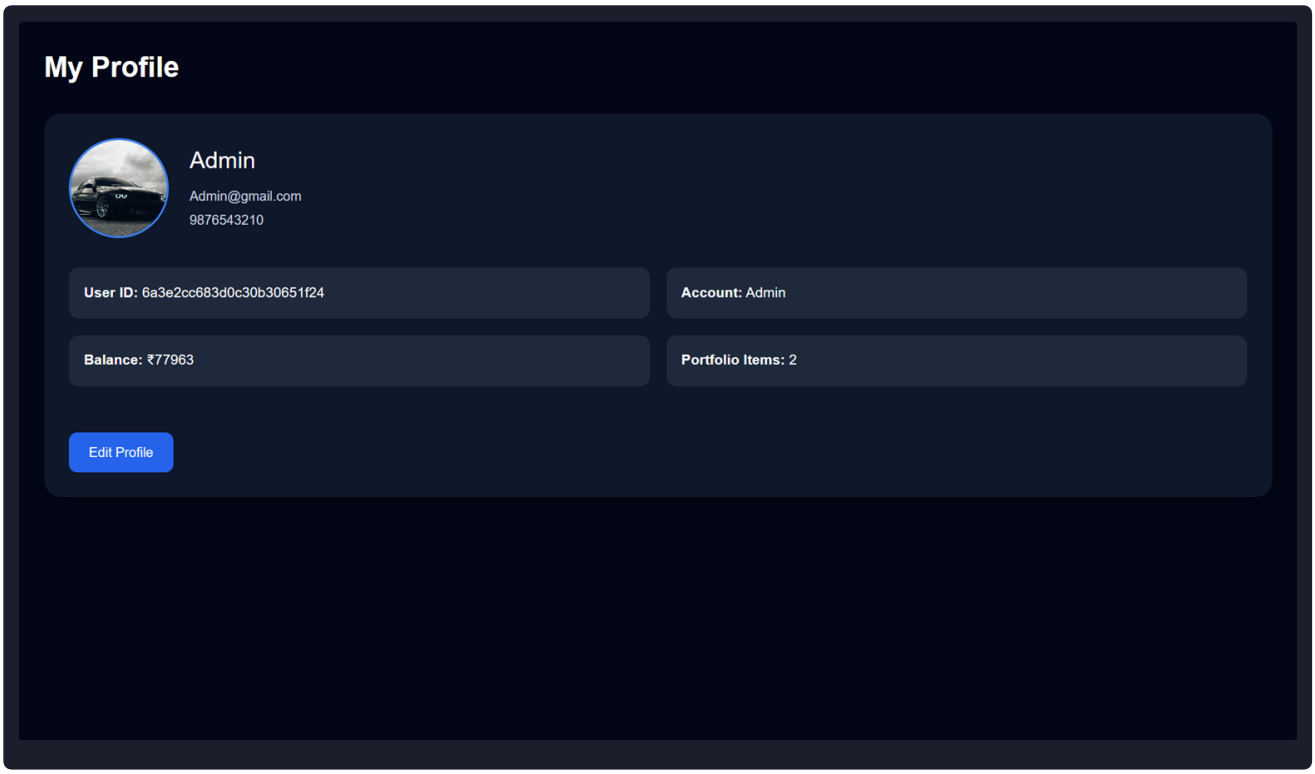
9.8 Watchlist

Displays the user's saved stocks for easy monitoring. Users can track stock information and remove stocks from the watchlist whenever required.



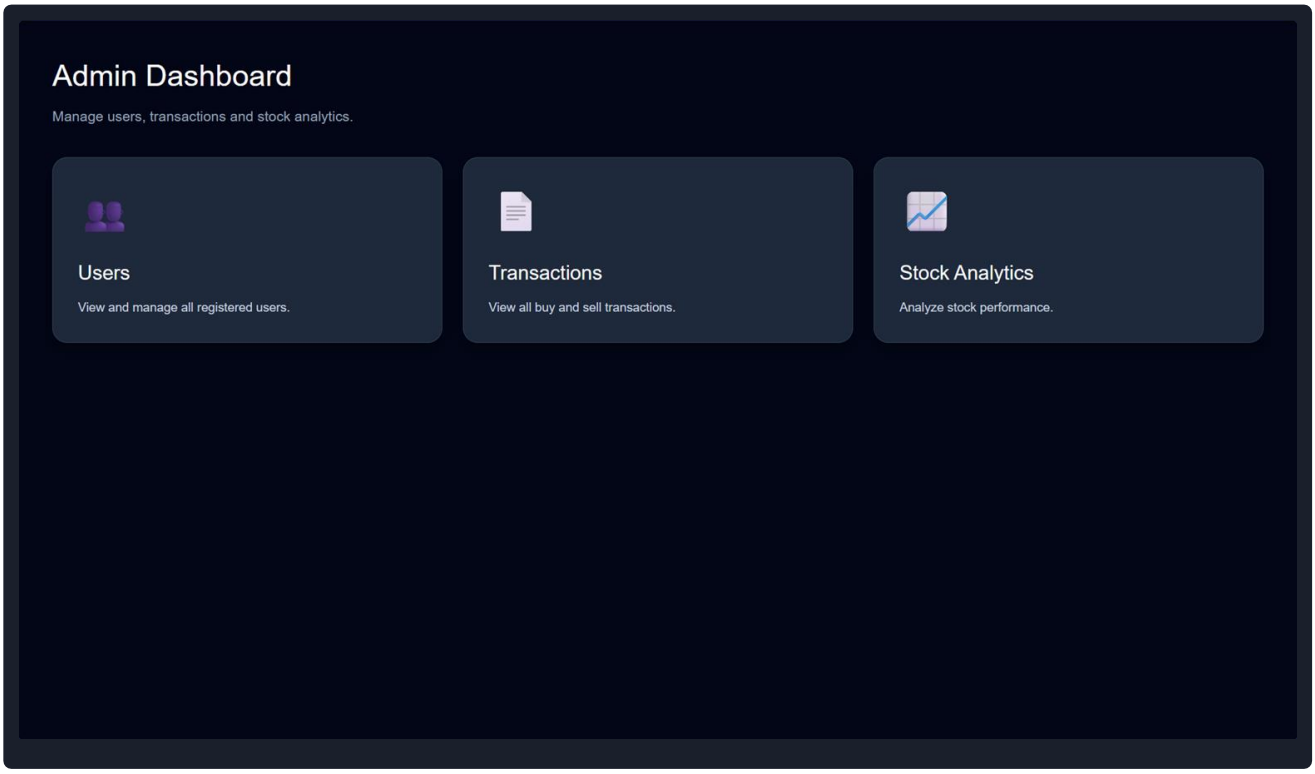
9.9 My Profile

Allows users to view and manage their personal account information, including profile details, email address, wallet balance, and account role. Users can also update their profile information and security settings.



9.10 Admin Dashboard

Displays an overview of the stock trading platform, including summary statistics such as total users, stocks, and transactions. It also provides quick access to administrative modules for managing users, stocks, orders, and analytics.



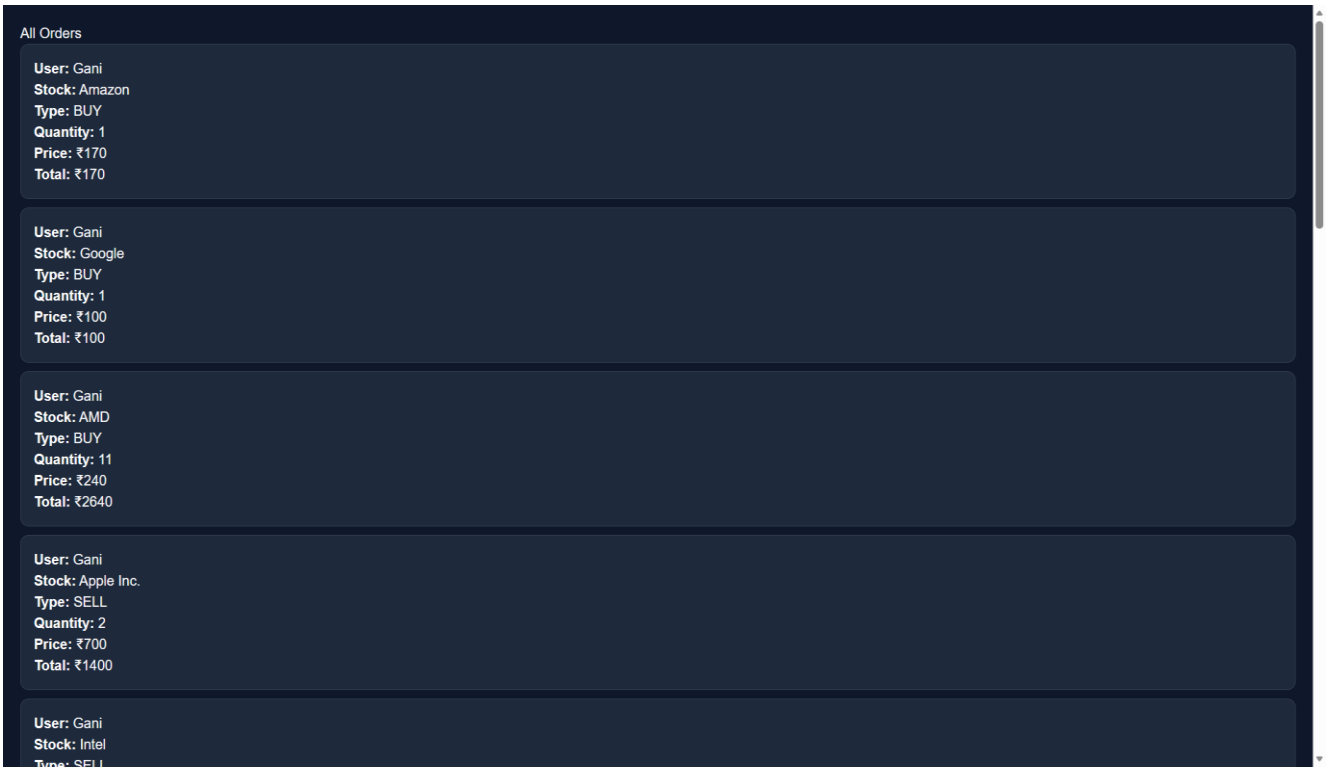
9.11 Stock Analytics Dashboard

Allows administrators to analyze stock performance using interactive charts and summary metrics. It provides insights into price trends and overall market activity to support platform monitoring.

Stock	BUY	SELL	Quantity	Total Value
AAPL - Apple Inc.	2	1	10	₹1,800
TSLA - Tesla Inc.	4	1	13	₹3,250

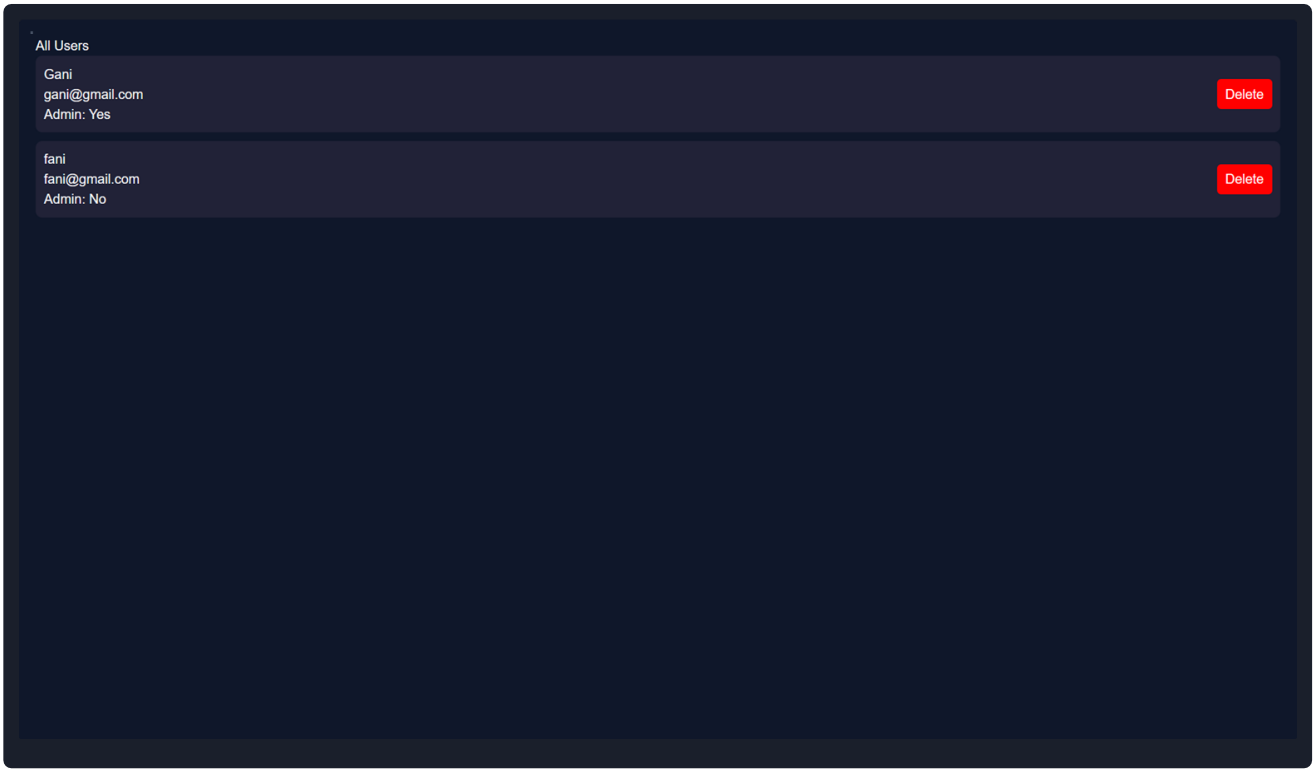
9.12 Global Order Analytics Dashboard

Provides administrators with graphical insights into all user orders and trading activity. The dashboard displays charts and summary metrics to monitor transaction volumes and overall platform performance



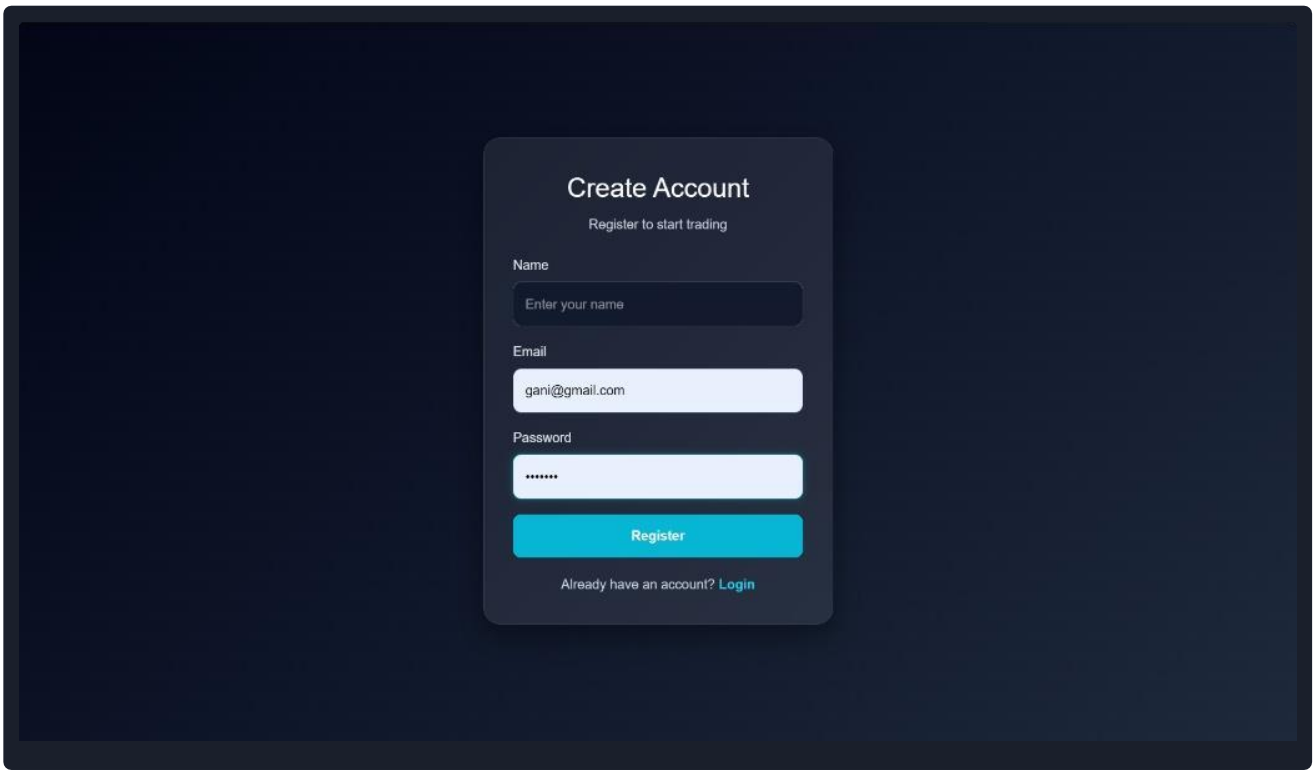
9.13 User Management

Allows administrators to view all registered users, check their names, email addresses, and admin status, and delete user accounts from the system.



9.14 User Registration Page

Allows new users to create an account by entering the required details. Upon successful registration, a user profile is created and initialized with virtual funds, enabling access to the stock trading platform.



10. TESTING

System validation was performed throughout the implementation process to ensure secure and reliable application behavior. Manual testing was used to verify user workflows, while Thunder Client was used to test REST API endpoints within the IDE. These validation routines confirmed request processing boundaries, authenticated user sessions, and ensured proper communication between the REST controllers and other application components.

Testing Parameter Track	Verification Description & Validation Activity
Manual UI Testing	Tested all application pages manually to ensure smooth navigation, proper form validation, button functionality, and correct page transitions.
REST API Testing	Verified all backend APIs using Thunder Client by testing GET, POST, PUT, and DELETE requests with valid and invalid data.
Authentication Testing	Checked user login, JWT token generation, protected routes, and ensured unauthorized users could not access restricted resources.
Database Testing	Verified that user data, stock details, portfolios, and transactions were correctly stored, updated, and retrieved from MongoDB Atlas.
Input Validation Testing	Tested the application with empty fields, incorrect inputs, and invalid data formats to ensure proper validation messages and error handling.
Admin Module Testing	Verified that only administrators could access admin features such as managing users, stocks, and transactions, while normal users were denied access.

11. INTERFACE IMPLEMENTATION MATRIX

The following operational presentation index maps core tracking views with corresponding architectural attributes:

Application View	Functional Description
Public Landing Platform	The primary presentation gate providing clear access parameters for visitors.
Login Page	Validates profile criteria using secure hashed key streams.
Registration Page	Seeds client profile instances and initial mock funds fields.
Dashboard	Unified entry workspace presenting financial analytics summary metrics.
Market Page	The core trading platform for buying, selling, or tracking assets.
Stock Analytics	Interactive graphing charts showing price trends.
Portfolio	Tracks individual stock holdings, net values, and real-time profit and loss metrics.
Transaction History	Historical ledger recording chronological trade data.
Watchlist	Monitors targeted stock changes to enable quick trading shortcuts.

Application View	Functional Description
Profile Management	Allows users to update their personal information, change passwords, and upload profile pictures securely.
Admin Dashboard	Main control deck for managing core marketplace systems.
Stock Management	Enables administrators to add, update, and remove stock listings while maintaining market data.

12. KNOWN SYSTEM LIMITATIONS

- **Simulated Stock Data:** The application uses predefined stock prices instead of live market data from external financial APIs.
- **No Email Verification:** User registration does not include email verification or account activation through SMTP services.
- **No Password Recovery:** The system does not currently support "Forgot Password" or password reset functionality.
- **Single-Factor Authentication:** User authentication relies only on a username and password without Two-Factor Authentication (2FA).
- **Continuous Atlas Availability Dependency:** Runtime stability depends on continuous internet access to preserve connection loops targeting cloud MongoDB Atlas servers.
- **Basic Analytics:** Dashboard visualizations are limited to the currently implemented charts and summary metrics..

13. FUTURE ENHANCEMENTS

- **Live Market Data Integration:** Connect the application with financial APIs such as Alpha Vantage or Yahoo Finance to provide real-time stock prices and market updates.
- **AI-Based Investment Suggestions:** Implement machine learning models to analyze user trading history and provide personalized investment recommendations.
- **Advanced Portfolio Analytics:** Add more detailed charts, performance metrics, and interactive dashboards for better portfolio analysis.
- **Email Verification and Password Recovery:** Integrate secure email services for account verification, password reset, and other account-related notifications.
- **Price Alert Notifications:** Notify users when watchlist stocks reach user-defined price targets through push or email notifications**.**
- **Mobile Application:** Develop a mobile version of the platform using React Native for Android and iOS devices.
- **Report Export Feature:** Allow users to export transaction history and portfolio details in PDF or Excel format for record keeping.